

データベース演習

第6回：SQLによるデータ分析

鈴木源吾

前回の復習

1. 解説：要件定義と概念設計
2. 例題：関連とテーブル
3. 演習：関連とテーブル

今回のトピック

1. 解説：SQLによるデータ分析
 - 現実的な分析の方法とは
 - 分析SQLの世界
2. 例題：サンプルデータのSQLによる検索
 - サンプルデータベースと2つの実行環境（Colab / pgAdmin）
 - SQLによる検索
3. 演習：サンプルデータのSQLによる検索

参考資料：その他の資料「10年戦えるデータ分析入門」第1章，第3章

1. 解説：SQLによるデータ分析

企業におけるデータ分析と課題

企業において、データ活用や分析というキーワードは今でも色あせていない
企業は保有する様々なデータを活用したい動機はあるがなかなか難しい

課題の例

- Excelで分析を始めたが、データがこまぎれで面倒
- 10万行を超えたあたりからExcelが遅くてどうにもならない
- 業務に近く分析したい人 → 分析したい要求はあるが技術がない
- 情報システムの専門家 → 技術はあるが、業務に詳しくない

→ **分析したい人とシステムを作っている人が、同じ方向を向くことが必要**

現実的な分析の方法：SQLによる分析

(Excelを超えて) 向かうべき方向は？・・・

SQLの使えるデータベースを使ってデータを分析する

なぜか？ → 以下の3つが理由としてあげられる

1. 企業の業務データはRDBMSに、分析用の大規模データはクラウドDWH（例：BigQuery、Redshift）に保存されている
2. 大規模データにも対応可能（クラウドDWHや分散処理基盤の活用）
3. 技術職・非技術職を問わず、多くの関係者が理解・活用できる共通言語（＝SQL）

1. 企業のデータはRDBMSやクラウドDWHにある

企業のデータは、主に以下のような基盤に蓄積されている。

- 業務システム → 多くはRDBMSを用いて構築されており、SQLが利用可能
- ビッグデータ（アクセスログやセンサーデータなど）
 - 以前：Hadoop（分散処理技術により、大規模データの蓄積と分析を可能にするオープンソースの基盤）の使用が多かった
 - 現在：SparkやクラウドベースのDWH（BigQuery, Snowflakeなど）へと移行が進んでいる
 - Hadoop, Spark, クラウドDWHでもSQLによるデータ分析が可能である

→ SQLが使えることで、様々な環境のデータにアクセスできるようになる

2. 大規模データにも対応可能

SQLによる分析は、扱うデータサイズに応じて柔軟に拡張できるという特徴を持つ。

- 小規模データからでも手軽に始められ、同じSQLで大規模データにも対応可能
- 100万件、1000万件、それ以上のビッグデータにも同様の構文で処理できる

処理基盤の違いによる対応力の比較：

- Excel：行数の上限は約100万行．PCのメモリにも依存する
- RDBMS（PostgreSQL，MySQLなど）：1000万行規模は一般的に処理可能
- 並列RDBMSやクラウドDWH：さらに大規模なデータにも対応可能

→ SQLは規模の大小を問わず、同じ言語での分析を可能にする

3. 様々な関係者間の共通言語として優れている

分析を必要とする人は多い。

例) 企画や商品開発を担う職種，分析を専門とするデータアナリスト

→ 参考書ではこれらを「プランナー」と呼んでいる

- 分析すべき目的や問い → プランナーが最も把握している
- 分析を実行する技術や環境 → エンジニアが担っている

→ したがって，チームでの協調的な分析が求められる

→ SQLはエンジニアだけでなくプランナーも習得可能で，共通言語として機能する

- SQLの学習ハードルはそれほど高くなく，理解できるプランナーも一定数存在
- SQLを使える技術者が多いことも，導入のしやすさにつながっている

SQLによる分析SQLの例

SQLだけで，Webアクセスの簡単な分析ができる

- この例は2014年11月16日～23日の期間にセールのページへのアクセスがどの程度か調べている

```
select
  count(*)
from
  access_log
where
  request_time between date '2014-11-16' and date '2014-11-23'
  and request_path = '/sale201411'
;
```

行の数を求めよ

アクセスログの
テーブルから

2014年11月16日～23日
の期間に

/sale201411というページ
にアクセスした

情報システムの世界と分析の世界

情報システムは大きく2つの世界に分けることができる

1. 基幹系

- 通常業務を回すためのシステムを指す
- いわゆる業務システムやWebアプリケーション
- オンライン：ユーザがリアルタイムに回線の向こうにいる
- トランザクション：商品注文，チケット予約，送金などの取引
- OLTPシステムと呼ばれる（online transaction processing）
 - 小さな取引（トランザクション）をリアルタイムにその場で大量に処理するシステム

情報システムの世界と分析の世界

2. 情報系

- 分析処理を行うためのシステムを指す
- OLAPシステムと呼ばれる（online analytical processing）
- 大量の集計処理が発生することが特徴
- 例）1日ごとの予約件数を数える，ユーザの年齢層毎の割合を計算する
- 新たなデータの書き込みがほとんどない（読むだけ）
- 処理時間が長くかかる可能性がある

項目	OLTP	分析処理
アクセスパターン	読み書き	読み込みのみ
1回のアクセス量	少ない	多い
アクセス数	多い	少ない

情報系もSQLで

RDBMSは本来，OLTP（オンライン処理）を想定して設計されたものであり，分析処理には不向きとされていた．

- 「行」単位の処理に最適化され，「列」単位での集計処理には弱い構造であった
- MySQLは分析処理に不向きと言われていたが，近年は機能強化により，PostgreSQLとの差は縮まりつつある

現在では，分析特化型のDBMSもSQLベースが主流となっている．

- 例：Redshift，BigQuery，Snowflake，Greenplum，Teradata など
- いずれも並列処理技術を活用し，大量データの高速分析を実現
- かつてSQL非対応のものも存在→汎用性や技術者層の広さからSQLベースへと移行

→ 現在，SQLは分析のあらゆる場面に応用可能な基盤技術である

2. 例題：サンプルデータのSQLによる検索

使用するサンプルデータベースについて

その他資料「青木峰郎, 10年戦えるデータ分析入門, ソフトバンククリエイティブ」のサンプルデータベースを使用する（その他資料は現在電子書籍版のみ）

- 元データはサポートページ <https://i.loveruby.net/stdsql/> で公開されている
- 本演習では，独自にデータを追加した **weblogデータベース** として提供する
- このデータベースは **ColabとpgAdminの2つの環境** で利用でき，どちらで実行してもよい
- 学生側でのデータベース作成・リストア作業は不要

2つの実行環境（Colab と pgAdmin）

weblogデータベースは，以下のどちらの環境でも実行できる

	Colab (SQLite)	pgAdmin (Supabase)
接続先	ノートブックがDLする <code>weblog.sqlite3</code>	DB基礎で使ったSupabase (postgres)
収録テーブル	customers / shops / items（軽量版）	weblogスキーマの全テーブル
準備	ノートブックのセルを実行するだけ	pgAdminでSupabaseに接続

- **例題・演習はどちらの環境で解いてもよい**（使うテーブルは両方にある）
- **ただし 課題の提出は Colab の演習ノートブック で行う**（→ 3章）

pgAdmin (Supabase) を使う場合

pgAdminで実行したい場合の接続と準備 (Colabで実行する場合は不要)

- pgAdminを起動し，DB基礎で設定したSupabaseサーバに接続する
 - 接続設定がない場合，DB基礎のマニュアルに従い設定する
- サーバ→データベース→**postgres**→スキーマ→**weblog** と展開する
- テーブルは `weblog.テーブル名` で参照する
 - またはクエリツールで最初に `SET search_path TO weblog, public;` を実行すれば，以降スキーマ名を省略できる

サンプルデータベースの構成

- 架空のECサイトのデータが入っている（18個のテーブル）
- テーブルの一部の内容を下表に示す
 - マスター:システムの運用上変更が頻繁にない基本的な情報のテーブル

テーブル名	内容
customers	顧客マスター
shops	店舗マスター
items	商品マスター
orders	商品の注文
order_details	商品の注文詳細（各注文で購入した商品と個数）
access_log	EC サイトのアクセスログ
access_log_wide	EC サイトのアクセスログ（カラムを増やしたバージョン）

SQLのおさらい

- 復習も含めてSQLをおさらいしていく（一部、新規の機能も含む）
- おさらいでは **access_logテーブル** を例として使う（下表がタプルの例）
 - access_logはデータ量が大きく、**Colab軽量版には含まれない**
 - このパートは **教員がpgAdmin（Supabase）で実演** する
 - 例題・演習（customers / items）は、引き続きどちらの環境でも実行できる

	request_time timestamp without time zone 🔒	request_method text 🔒	request_path text 🔒	customer_id integer 🔒	search_hit integer 🔒	referer text 🔒
1	2012-01-01 00:00:02	POST	/search	59856	38	[null]
2	2012-01-01 00:02:45	GET	/items/218	59856	[null]	/search
3	2012-01-01 00:04:07	POST	/search	59856	25	/items/218
4	2012-01-01 00:01:43	GET	/	79868	[null]	http://goo...
5	2012-01-01 00:03:20	POST	/search	84526	26	[null]

基本的なデータの抽出：FROM句・SELECT句

テーブル・カラムを指定して，全行を抽出する

例：SELECT request_time, customer_id FROM access_log;

- FROMの後に検索したいテーブルを指定する（FROM句）
- SELECTの後に検索したいカラムを指定する（SELECT句）
 - 例は，テーブル access_log，カラム request_time, customer_idを指定

全カラムを抽出する

例：SELECT * FROM access_log;

- SELECT句に「*」を指定すると，すべてのカラムを指定したことになる

WHERE句による行の絞り込み

基本的な絞り込み

```
例 : SELECT * FROM access_log  
      WHERE request_time >= timestamp '2014-12-29 00:00:00';
```

- WHEREの後に絞り込みたい条件を指定する（WHERE句）
 - 例は，request_timeの値が2014年12月29日0時より大きい（＝以降である）行に絞り込まれる）
- 条件式の両辺のデータ型は同じである必要がある
 - 左辺のカラム request_timeは，timestamp型である
 - 右辺もtimestamp型にするために，文字列'2014-12-29 00:00:00'の前に型変換（キャスト）を示すtimestampを指定している
 - 「CAST('2014-12-29 00:00:00' AS timestamp)」と指定してもよい

WHERE句による行の絞り込み

条件式の組み合わせ

```
例 : SELECT * FROM access_log  
      WHERE request_time >= timestamp '2014-12-29 00:00:00'  
      AND request_time < timestamp '2014-12-30 00:00:00';
```

- AND：かつ，OR：または，を使って複数の条件式を組み合わせできる
 - 例は「2014年12月29日0時以降」かつ「2014年12月30日0時より前」の行を返す

WHERE句による行の絞り込み

複数の候補による絞り込み

1つのカラムの値がXかYかZだったら抜き出すのはよく使う．複数の条件式をORで組み合わせればできる

```
例 : SELECT * FROM access_log WHERE customer_id = 59856  
      OR customer_id = 79868  
      OR customer_id = 71941;
```

- これはカラム customer_idの値が、「59856」または「79868」または「71941」である行だけを返す

これは、IN演算子を使う方法に書き換えることができる．かなりすっきり書ける

```
例 : SELECT * FROM access_log WHERE customer_id IN (59856, 79868, 71941);
```

WHERE句による行の絞り込み

文字列のパターンによる絞り込み

```
例 : SELECT * FROM access_log  
      WHERE request_path LIKE '/items%';
```

- LIKE演算子は文字列パターンマッチに使う
- 「%」は「ここはなんでもいい」という意味を表す
- 上記は、/itemsという文字列で始まるすべての文字列にマッチングする
- 下表のようなパターンがあるが、%を2つ以上使っても構わない

パターン	意味
<i>str</i> %	<i>str</i> で始まる
% <i>str</i>	<i>str</i> で終わる
% <i>str</i> %	<i>str</i> をどこかに含む
<i>str</i> % <i>str</i> 2	<i>str</i> で始まり <i>str</i> 2 で終わる

WHERE句による行の絞り込み

NOT句で条件を反転

```
例 : SELECT count(*) FROM access_log  
      WHERE NOT request_method = 'POST';
```

- これは、request_methodカラムの値が文字列POST「ではない」行の数を返す
- NOTをつけることによって、条件式の真偽を反転することができる
- request_methodカラムの値は、GETとPOSTの2種類しかないので、この行の数は、request_methodカラムの値がGETであるような行の数に一致する．確かめてみよう
- 参考：SELECT DISTINCT request_method FROM access_log; でrequest_methodカラムの値のリストを得ることができる

行数を求める・結果制限

行数を求める

例 : `SELECT count(*) FROM access_log;`

- `count()`は行の数を返却する集約関数で、この問合せは全行数を返す

結果行数の指定

例 : `SELECT * FROM access_log LIMIT 3;`

- `LIMIT`句を使うと結果行数を指定できる．この例では3行だけが返る
- 100万行あるようなテーブルには有効な機能

ORDER BY句による行の並び替え

行を並び替える

```
例 : SELECT * FROM access_log  
      ORDER BY request_time;
```

- これは、request_timeの小さい順（つまり古い順）に並べている

並び順を制御する

```
例 : SELECT * FROM access_log  
      ORDER BY request_time DESC;
```

- 大きい順（この場合新しい順）にするためには、最後にDESCを追加する（降順 descendingの略．昇順はASC． ascendingの略）

ORDER BY句による行の並び替え

複数の並び替え条件を指定する

```
例 : SELECT * FROM access_log  
      ORDER BY customer_id, request_time;
```

- このように複数のカラムを指定するときは、まず、行がcustomer_idの小さい順に並び替えられ、同じcustomer_idの行の中ではrequest_timeが小さい順に並び替えられる

```
例 : SELECT * FROM access_log  
      ORDER BY customer_id DESC, request_time ASC
```

というカラムごとに降順・昇順を指定することも可能
(この例で、ASCはデフォルト値であるので、省略しても結果は同じ)

複数の句の組み合わせ

今までに学んだ様々な機能を組み合わせると以下のようなSQLも書ける

```
例 : SELECT request_time, customer_id FROM access_log  
      WHERE request_time >= timestamp '2014-12-29 00:00:00'  
        AND request_time < timestamp '2014-12-30 00:00:00'  
      ORDER BY request_time  
      LIMIT 3;
```

- 「access_logテーブルから、2014年12月29日だけのログを抜き出し、古い順に3行みたい、カラムはrequest_timeとcustomer_idだけでよい」という条件
- 句の順番はこのままでなければいけない．ORDER BY句は、WHERE句の前には書けない．WHERE句はSELECT句の後

例題

customersテーブルに対して、以下のSQL文を作成し、実行せよ（Colab・pgAdminのどちらの環境でもよい．行数が少し多いため、(3)-(5)は指定された数に結果行数を制限すること）

まずは次ページ以降の解答を見ずに、自力で書いてみよう

- (1) customersテーブルの全行数を数えるSQL文
- (2) customer_genderがFである行数を数えるSQL文（なお、Fは女性を表す）
- (3) customer_ageが30より大きい行の、customer_idとcustomer_ageを求めるSQL文（結果の上限を10行に制限せよ）
- (4) customer_ageが26または19である行の、customer_idとcustomer_ageを求めるSQL文（結果の上限を10行に制限せよ）
- (5) customer_ageが若い順に、customer_idとcustomer_ageを求めるSQL文（結果の上限を10行に制限せよ）

例題の解答

(1) customersテーブルの全行数を数えるSQL文

```
SELECT count(*) FROM customers;
```

(2) customer_genderがFである行数を数えるSQL文 (Fは女性を表す)

```
SELECT count(*) FROM customers WHERE customer_gender = 'F';
```

(3) customer_ageが30より大きい行の, customer_idとcustomer_ageを求めるSQL文
(結果の上限を10行に制限せよ)

```
SELECT customer_id, customer_age FROM customers WHERE customer_age > 30 LIMIT 10;
```

例題の解答

(4) customer_ageが26または19である行の, customer_idとcustomer_ageを求めるSQL文(結果の上限を10行に制限せよ)

```
SELECT customer_id, customer_age FROM customers WHERE customer_age = 26 OR customer_age = 19 LIMIT 10;
```

(5) customer_ageが若い順に, customer_idとcustomer_ageを求めるSQL文(結果の上限を10行に制限せよ)

```
SELECT customer_id, customer_age FROM customers ORDER BY customer_age LIMIT 10;
```


3. 演習：サンプルデータのSQLによる検索

- 演習も Colab・pgAdmin のどちらの環境で解いてもよい
- ただし で行う
 - 解答SQLと実行結果をノートブックに残し、そのリンクを提出する
- WebClassの提出フォームは次回までに準備する（今回・次回のノートブックが対象）

演習

itemsテーブルに対して、以下のようなSQL文を作成せよ（行数が少し多いため、(3)-(5)は指定された数に結果行数を制限すること）

- (1) itemsテーブルの全行数を数えるSQL文
- (2) item_priceが10000より大きい行数を数えるSQL文
- (3) item_priceが2000より小さい行の、item_idとitem_priceを求めるSQL文(結果の上限を10行に制限せよ)
- (4) shop_idが1または2である行の、item_idとshop_idを求めるSQL文(結果の上限を10行に制限せよ)
- (5) item_priceが大きい順に、item_idとitem_priceを求めるSQL文(結果の上限を10行に制限せよ)